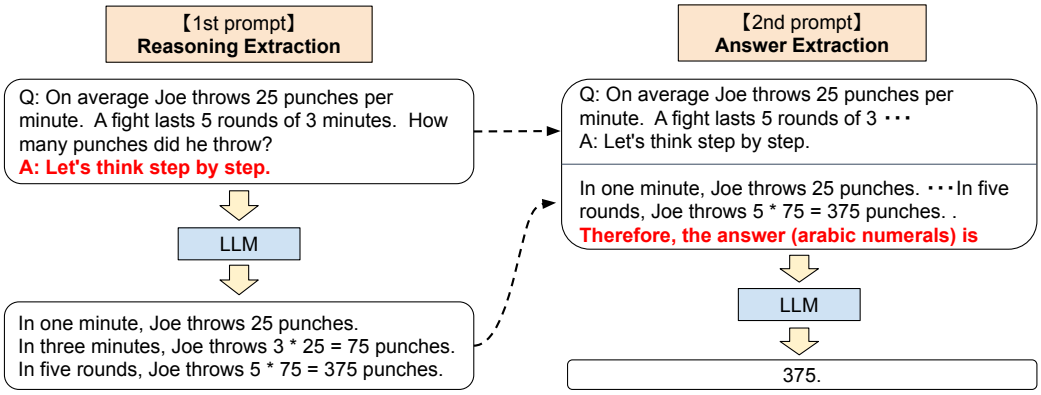

Figure 2: Full pipeline of Zero-shot-CoT as described in § 3: we first use the first "reasoning" prompt to extract a full reasoning path from a language model, and then use the second "answer" prompt to extract the answer in the correct format from the reasoning text.

in Figure 1). In summary, Few-shot-CoT [Wei et al., 2022] requires careful human engineering of a few prompt examples with specific answer formats per task, while Zero-shot-CoT requires less engineering but requires prompting LLMs twice.

1st prompt: reasoning extraction In this step we first modify the input question $\mathbf{x}$ into a *prompt* $\mathbf{x}'$ using a simple template "Q: `[X]`. A: `[T]`", where `[X]` is an input slot for $\mathbf{x}$ and `[T]` is an slot for hand-crafted trigger sentence $\mathbf{t}$ that would extract chain of though to answer the question $\mathbf{x}$. For example, if we use "Let's think step by step" as a trigger sentence, the prompt $\mathbf{x}'$ would be "Q: `[X]`. A: Let's think step by step.". See Table 4 for more trigger examples. Prompted text $\mathbf{x}'$ is then fed into a language model and generate subsequent sentence $\mathbf{z}$. We can use any decoding strategy, but we used greedy decoding throughout the paper for the simplicity.

2nd prompt: answer extraction In the second step, we use generated sentence $\mathbf{z}$ along with prompted sentence $\mathbf{x}'$ to extract the final answer from the language model. To be concrete, we simply concatenate three elements as with "`[X']` `[Z]` `[A]`": `[X']` for 1st prompt $\mathbf{x}'$, `[Z]` for sentence $\mathbf{z}$ generated at the first step, and `[A]` for a trigger sentence to extract answer. The prompt for this step is *self-augmented*, since the prompt contains the sentence $\mathbf{z}$ generated by the same language model. In experiment, we use slightly different answer trigger depending on the answer format. For example, we use "Therefore, among A through E, the answer is" for multi-choice QA, and "Therefore, the answer (arabic numerals) is" for math problem requiring numerical answer. See Appendix A.5 for the lists of answer trigger sentences. Finally, the language model is fed the prompted text as input to generate sentences $\hat{\mathbf{y}}$ and parse the final answer. See "Answer Cleansing" at §4 for the parser details.

4 Experiment

Tasks and datasets We evaluate our proposal on 12 datasets from four categories of reasoning tasks: arithmetic, commonsense, symbolic, and other logical reasoning tasks. See Appendix A.2 for the detailed description of each datasets.

For arithmetic reasoning, we consider the following six datasets: (1) SingleEq [Koncel-Kedziorski et al., 2015], (2) AddSub [Hosseini et al., 2014], (3) MultiArith [Roy and Roth, 2015], (4) AQUA-RAT [Ling et al., 2017], (5) GSM8K [Cobbe et al., 2021], and (6) SVAMP [Patel et al., 2021]. The first three are from the classic Math World Problem Repository [Koncel-Kedziorski et al., 2016], and the last three are from more recent benchmarks. SingleEq and AddSub contain easier problems, which do not require multi-step calculation to solve the tasks. MultiArith, AQUA-RAT, GSM8k, and SVAMP are more challenging datasets that require multi-step reasoning to solve.

For commonsense reasoning, we use CommonsenseQA [Talmor et al., 2019] and StrategyQA [Geva et al., 2021]. CommonsenseQA asks questions with complex semantics that often require reasoning

based on prior knowledge [Talmor et al., 2019]. StrategyQA requires models to infer an implicit multi-hop reasoning to answer questions [Geva et al., 2021].

For symbolic reasoning, we use Last Letter Concatenation and Coin Flip [Wei et al., 2022]. Last letter Concatenation asks the model to concatenate the last letters of each word. We used randomly selected four names for each sample. Coin Flip asks the model to answer whether a coin is still heads up after people either flip or do not flip the coin. We created samples of four times flip or not flip trials. Although these tasks are easy for humans, LMs typically exhibit a flat scaling curve.

For other logical reasoning tasks, we choose two evaluation sets from the BIG-bench effort [Srivastava et al., 2022]: Date Understanding [2] and Tracking Shuffled Objects. Date Understanding asks models to infer the date from a context. Tracking Shuffled Objects tests a model's ability to infer the final state of objects given its initial state and a sequence of object shuffling. We used a dataset of tracking three shuffled objects for our experiment.

Models We experiment with 17 models in total. Main experiments are conducted with Instruct-GPT3 [Ouyang et al., 2022] (text-ada/babbage/curie/davinci-001 and text-davinci-002)[3], original GPT3 [Brown et al., 2020] (ada, babbage, curie, and davinci)[4], and PaLM [Chowdhery et al., 2022] (8B, 62B, and 540B). In addition, we used GPT-2[Radford et al., 2019], GPT-Neo[Black et al., 2021], GPT-J[Wang and Komatsuzaki, 2021], T0 [Sanh et al., 2022], and OPT [Zhang et al., 2022] for model scaling study. The size of LMs ranges from 0.3B to 540B. We include both standard (e.g. GPT-3 and OPT), and instruction following variants (e.g. Instruct-GPT3 and T0). See Appendix A.3 for model description details. Unless otherwise stated, we use text-davinci-002 throughout the experiments.

Baselines We compare our Zero-shot-CoT mainly to standard Zero-shot prompting to verify the effectiveness of its chain of thought reasoning. For Zero-shot experiments, similar answer prompts as Zero-shot-CoT are used as default. See Appendix A.5 for detail. To better evaluate the zero-shot ability of LLMs on reasoning tasks, we also compare our method to Few-shot and Few-shot-CoT baselines from [Wei et al., 2022], using the same in-context examples. Throughout the experiments, we use greedy decoding across all the methods. For the zero-shot approaches, the results are therefore deterministic. For the few-shot approaches, since the order of in-context examples could affect the results [Lu et al., 2022], we run each experiment only once with a fixed seed across all methods and datasets, for fair comparisons with the zero-shot methods. Wei et al. [2022] showed that the order of examples did not cause large variance in CoT experiments.

Answer cleansing After the model outputs a text by answer extraction (see § 3 and Figure 2), our method picks up only the part of the answer text that first satisfies the answer format. For example, if the answer prompting outputs "probably 375 and 376" on arithmetic tasks, we extract the first number "375" and set it as the model prediction. In the case of multiple-choice, the first large letter we encounter is set as the prediction. See Appendix A.6 for more detail. Standard Zero-shot method follows the same idea. For Few-shot and Few-shot-CoT methods, we follow [Wang et al., 2022] and first extract the answer text after "The answer is " from the model output, and apply the same answer cleansing to parse the answer text. If "The answer is" is not found in the model output, we search from the back of the text and set the first text that satisfies the answer format as the prediction.

4.1 Results

Zero-shot-CoT vs. Zero-shot Table 1 summarize accuracy of our method (Zero-shot-CoT) and standard zero-shot prompting (Zero-shot) for each dataset. Zero-shot-CoT substantially outperforms four out of six arithmetic reasoning tasks (MultiArith, GSM8K, AQUA, SVAMP), all symbolic reasoning, and all other logical reasoning tasks (from BIG-bench [Srivastava et al., 2022]). For

[2] While prior work [Wei et al., 2022] categorized Date Understanding task into Common Sense reasoning, our study categorized this task into logical reasoning because this task requires less prior knowledge and more logical reasoning between dates.

[3] Our experiment for Instruct GPT-3 models includes both text-****-001 and text-davinci-002. Text-davinci-002 differs from text-****-001 in that they use different fine-tuning data depending on the date range collected from the APIs. Specifically, text-davinci-002 uses data up to Jun 2021, while text-****-001 uses data up to Oct 2019. (See `https://beta.openai.com/docs/engines/gpt-3`)

[4] Our experiments with GPT3 series are conducted by using OpenAI API between April-2022 and May-2022, except for No.10-16 in Table 4 in Aug-2022.

Table 1: Accuracy comparison of Zero-shot-CoT with Zero-shot on each tasks. The values on the left side of each task are the results of using answer extraction prompts depending on answer format as described at § 3. The values on the right side are the result of additional experiment where standard answer prompt "The answer is" is used for answer extraction. See Appendix A.5 for detail setups.

	Arithmetic					
	SingleEq	AddSub	MultiArith	GSM8K	AQUA	SVAMP
zero-shot	74.6/**78.7**	**72.2/77.0**	17.7/22.7	10.4/12.5	22.4/22.4	58.8/58.7
zero-shot-cot	**78.0/78.7**	69.6/74.7	**78.7/79.3**	**40.7/40.5**	**33.5/31.9**	**62.1/63.7**
	Common Sense		Other Reasoning Tasks		Symbolic Reasoning	
	Common SenseQA	Strategy QA	Date Understand	Shuffled Objects	Last Letter (4 words)	Coin Flip (4 times)
zero-shot	**68.8/72.6**	12.7/**54.3**	49.3/33.6	31.3/29.7	0.2/-	12.8/53.8
zero-shot-cot	64.6/64.0	**54.8**/52.3	**67.5/61.8**	**52.4/52.9**	**57.6**/-	**91.4/87.8**

Table 2: Comparison with baseline methods using accuracies on MultiArith and GSM8K. text-davinci-002 is used as the model if not specified. We used the same 8 examples as described in [Wei et al., 2022] for Few-shot and Few-shot-CoT settings. (*1) To verify the variance of changing examples, we report two results for 4-shot-cot by splitting the eight examples into two groups. (*2) We insert "Let's think step by step." at the beginning of answer part of each exemplars for Few-shot-CoT to test performance gains. Further experiment results with PaLM are found at Appendix D

	MultiArith	GSM8K
Zero-Shot	**17.7**	**10.4**
Few-Shot (2 samples)	33.7	15.6
Few-Shot (8 samples)	33.8	15.6
Zero-Shot-CoT	**78.7**	**40.7**
Few-Shot-CoT (2 samples)	84.8	41.3
Few-Shot-CoT (4 samples : First) (*1)	89.2	-
Few-Shot-CoT (4 samples : Second) (*1)	90.5	-
Few-Shot-CoT (8 samples)	93.0	48.7
Zero-Plus-Few-Shot-CoT (8 samples) (*2)	**92.8**	**51.5**
Finetuned GPT-3 175B [Wei et al., 2022]	-	33
Finetuned GPT-3 175B + verifier [Wei et al., 2022]	-	55
PaLM 540B: Zero-Shot	**25.5**	**12.5**
PaLM 540B: Zero-Shot-CoT	**66.1**	**43.0**
PaLM 540B: Zero-Shot-CoT + self consistency	**89.0**	**70.1**
PaLM 540B: Few-Shot [Wei et al., 2022]	-	17.9
PaLM 540B: Few-Shot-CoT [Wei et al., 2022]	-	56.9
PaLM 540B: Few-Shot-CoT + self consistency [Wang et al., 2022]	-	74.4

example, Zero-shot-CoT achieves score gains from 17.7% to 78.7% on MultiArith and from 10.4% to 40.7% on GSM8K. Our method gives on-par performances for the remaining two arithmetic reasoning tasks (SingleEq and AddSub), which is expected since they do not require multi-step reasoning.

In commonsense reasoning tasks, Zero-shot-CoT does not provide performance gains. It is expected as Wei et al. [2022] also reports that even Few-shot-CoT does not provide performance gains on Lambda (135B), but does improve StrategyQA when combined with substantially larger PaLM (540B) model, which may also apply for ours. More importantly, we observe that many generated chain of thought themselves are surprisingly *logically* correct or only contains human-understandable mistakes (See Table 3), suggesting that Zero-shot-CoT does elicit for better commonsense reasoning even when the task metrics do not directly reflect it. We provide samples generated by Zero-shot-CoT for each dataset in Appendix B.